

Tarea 03

Nombre: David Alejandro Díaz Pineda

1. Utilice aritmética de corte de tres dígitos para calcular las siguientes sumas. Para cada parte, ¿qué método es más preciso y por qué?

```
In [1]: import math
import numpy as np
```

Definimos una función para truncar a n cifras significativas

```
In [2]: def truncar_cifras_significativas(x, n):
    if x == 0:
        return 0
    orden = math.floor(math.log10(abs(x)))
    factor = 10 ** (orden - n + 1)
    return math.trunc(x / factor) * factor
```

Definimos la función de sumatoria para calcular el valor real

```
In [3]: def sumatoria_real(funcion, inicio, fin):
    suma = 0
    for i in range(inicio, fin + 1):
        suma += funcion(i)
    return suma
```

```
In [4]: sum1 = sumatoria_real(lambda x: 1/x**2, 1, 10)
sum1
```

Out[4]: 1.5497677311665408

```
In [5]: sum2 = sumatoria_real(lambda x: 1/x**3, 1, 10)
sum2
```

Out[5]: 1.197531985674193

También definimos la función de sumatoria truncada en python

```
In [6]: def sumatoria(funcion, inicio, fin, inverso=False):
    suma = 0
    if inverso:
        rango = range(fin, inicio - 1, -1)
    else:
        rango = range(inicio, fin + 1)

    for i in rango:
```

```

    termino = truncar_cifras_significativas(funcion(i), 3)
    suma = truncar_cifras_significativas(suma + termino, 3)
    return suma

```

También definimos las funciones de error absoluto y error relativo

```

In [7]: def e_abs(aprox, real):
        return abs(aprox-real)

```

```

In [8]: def e_rel(aprox, real):
        return abs(aprox-real)/abs(real)

```

a)

$$\sum_{i=1}^{10} \left(\frac{1}{i^2} \right)$$

Primero por $\frac{1}{1} + \frac{1}{4} + \dots + \frac{1}{100}$ y luego por $\frac{1}{100} + \frac{1}{81} + \dots + \frac{1}{1}$

```

In [9]: parte1 = sumatoria(lambda x: 1/x**2, 1, 10)
        parte1

```

Out[9]: 1.53

```

In [10]: parte2 = sumatoria(lambda x: 1/x**2, 1, 10, inverso=True)
        parte2

```

Out[10]: 1.54

Con esto podemos calcular cual parte es más precisa

- Parte 1

```

In [11]: print(e_abs(parte1, sum1))

```

0.019767731166540736

```

In [12]: print(e_rel(parte1, sum1))

```

0.012755286336786206

$$E_{abs} = 1.97 * 10^{-2}$$

$$E_{rel} = 1.27 * 10^{-2} \approx 1.27\%$$

- Parte 2

```

In [13]: print(e_abs(parte2, sum1))

```

0.009767731166540727

```
In [14]: print(e_rel(parte2, sum1))
```

0.006302706508922058

$$E_{abs} = 9.76 * 10^{-3}$$

$$E_{rel} = 6.3 * 10^{-3} \approx 0.6\%$$

b)

$$\sum_{i=1}^{10} \left(\frac{1}{i^3} \right)$$

Primero por $\frac{1}{1} + \frac{1}{8} + \frac{1}{27} \dots + \frac{1}{27}$ y luego por $\frac{1}{100} + \frac{1}{729} + \dots + \frac{1}{1}$

```
In [15]: parte1 = sumatoria(lambda x: 1/x**3, 1, 10)
parte1
```

Out[15]: 1.16

```
In [16]: parte2 = sumatoria(lambda x: 1/x**3, 1, 10, inverso=True)
parte2
```

Out[16]: 1.19

Con esto podemos calcular cuál de los dos métodos es más preciso

- Parte 1

```
In [17]: print(e_abs(parte1, sum2))
```

0.037531985674193136

```
In [18]: print(e_rel(parte1, sum2))
```

0.03134111332572313

$$E_{abs} = 3.75 * 10^{-2}$$

$$E_{rel} = 3.13 * 10^{-2} \approx 3.13\%$$

- Parte 2

```
In [19]: print(e_abs(parte2, sum2))
```

0.007531985674193109

```
In [20]: print(e_rel(parte2, sum2))
```

0.00628959039449181

$$E_{abs} = 7.53 * 10^{-3}$$

$$E_{rel} = 6.28 * 10^{-3} \approx 0.628\%$$

El resultado más efectivo es en orden inverso, ya que los números "más pequeños" tienen el mismo orden de magnitud y así el truncamiento no provoca la pérdida de información tan rápido, cuando llega a los números grandes, los números más pequeños ya están sumados y no hay tanta pérdida de información

2. La serie de Maclaurin para la función arcotange converge para $-1 < x \leq 1$ y está dada por

$$\arctan x = \lim_{x \rightarrow \infty} P_n(x) = \lim_{n \rightarrow \infty} \sum_{i=1}^n (-1)^{i+1}$$

a) Utilice el hecho de que $\tan \pi/4 = 1$ para determinar el número n de términos de la serie que se necesita sumar para garantizar que $|4P_n(1) - \pi| < 10^{-3}$



$$|4P_n(1) - \pi| < \frac{4}{2n+1} < 10^{-3}$$

$$\frac{4}{2n+1} < 10^{-3}$$

$$2n+1 > \frac{4}{10^{-3}}$$

$$2n+1 > 4000$$

$$2n > 3999$$

$$n > 1999$$

Se necesitaría al menos 2000 terminos para poder llegar al error deseado.

b) El lenguaje de programación C++ requiere que el valor de π se encuentre dentro de 10^{-10} . ¿Cuántos términos de la serie se necesitarían sumar para obtener este grado de precisión?

$$\frac{4}{2n+1} < 10^{-10}$$

$$2n + 1 > \frac{4}{10^{-10}}$$

$$2m + 1 > 4x10^9$$

$$2n > 4x10^9 - 1$$

$$n > \frac{4x10^9 - 1}{2}$$

Se necesitarían de al menos 2000000000 de términos para llegar a una precisión de 10^{-10}

3. Otra fórmula para calcular π se puede deducir a partir de la identidad

$$\pi/4 = 4\arctan\frac{1}{5} - \arctan\frac{1}{239}.$$

Determine el número de términos que se deben sumar para garantizar una aproximación π dentro de 10^{-3}

```
In [21]: def arctan(x, n):
    suma = 0.0
    signo = -1
    for i in range(1, n+1):
        suma = suma + (signo ** (i+1)) * (x**(2*i-1)) / (2*i-1)
    return suma

tolerancia = 1e-3

n = 1

while True:
    pi_aprox = 16 * arctan(1/5, n) - 4 * arctan(1/239, n)

    error = abs(pi_aprox - 3.141592653589793)

    if error < tolerancia:
        break
    n = n+1

print(f"Número de términos : {n}")
```

Número de términos : 2

4. Compare los siguientes tres algoritmos. ¿Cuándo es correcto el algoritmo de la parte 1a?

a. ENTRADA $n, x_1, x_2, \dots, x_n$.

SALIDA PRODUCT.

Paso 1 Determine $PRODUCT = 0$.

Paso 2 Para $i = 1, 2, \dots, n$ haga

Determine $PRODUCT = PRODUCT * x_i$

Paso 3 SALIDA PRODUCT;

PARE.

b. ENTRADA $n, x_1, x_2, \dots, x_n$.

SALIDA PRODUCT.

Paso 1 Determine $PRODUCT = 1$.

Paso 2 Para $i = 1, 2, \dots, n$ haga

Set $PRODUCT = PRODUCT * x_i$.

Paso 3 SALIDA PRODUCT;

PARE.

c. ENTRADA $n, x_1, x_2, \dots, x_n$.

SALIDA PRODUCT.

Paso 1 Determine $PRODUCT = 1$.

Paso 2 Para $i = 1, 2, \dots, n$ haga

si $x_i = 0$ entonces determine $PRODUCT = 0$;

SALIDA PRODUCT;

PARE

Determine $PRODUCT = PRODUCT * x_i$.

Paso 3 SALIDA PRODUCT;

PARE.

El primer algoritmo siempre va a dar cero debido a que inicializa la variable producto en cero pero nunca la cambia y por tanto el resultado queda en cero. El segundo algoritmo siempre es una implementación correcta, mientras que el tercero es una forma mas optimizada del anterior debido a que verifica si existe algun cero dentro de la lista de input para terminar rapidamente la ejecución.

5.

a) ¿Cuántas multiplicaciones y sumas se requieren para determinar una suma de la forma

$$\sum_{i=1}^n \sum_{j=1}^i a_i b_j$$

Como hay una sumatoria anidada dentro de otra, se realizaran n multiplicaciones n veces lo que nos daría un total n^2 de multiplicaciones totales. Y como hay n^2 multiplicaciones se realizarían $n - 1$ sumas

b) Modifique la suma en la parte a) a un formato equivalente que reduzca el número de cálculos

$$\left(\sum_{i=1}^n a_i\right) \cdot \left(\sum_{j=1}^n b_j\right)$$

Discusiones

1. Escriba un algoritmo para sumar la serie finita $\sum_{i=1}^n x_i$ en orden inverso

- Entrada = $x_1, x_2, \dots, x_n$
- suma = 0
- Para i desde n hasta 1 con paso -1 hacer
 - suma = suma + $x[i]$
- FinPara
- Salida = suma

2. Las ecuaciones (1.2) y (1.3) en la sección 1.2 proporcionan formas alternativas para las raíces x_1 y x_2 de $ax^2 + bx + c = 0$. Construya un algoritmo con entrada a, b, c y salida x_1, x_2 que calcule las raíces x_1 y

x_2 (que pueden ser iguales pero con conjugados complejos) mediante la mejor fórmula para cada raíz.

```
In [22]: import cmath

def calcular_raices(a, b, c):
    discriminante = b**2 - 4 * a * c

    if discriminante > 0:
        if b > 0:
            x1 = (-b - discriminante**0.5) / (2 * a)
        else:
            x1 = (-b + discriminante**0.5) / (2 * a)
        x2 = c / (a * x1)
    elif discriminante == 0:
        x1 = x2 = -b / (2 * a)
    else:
        raiz_discriminante = cmath.sqrt(discriminante)
        x1 = (-b + raiz_discriminante) / (2 * a)
        x2 = (-b - raiz_discriminante) / (2 * a)

    return x1, x2

a, b, c = 1, -3, 2
raiz1, raiz2 = calcular_raices(a, b, c)
print(f"Las raíces son: x1 = {raiz1}, x2 = {raiz2}")
```

Las raíces son: x1 = 2.0, x2 = 1.0

3. Suponga que

$$\frac{1 - 2x}{1 - x + x^2} + \frac{2x - 4x^3}{1 - x^2 + x^4} + \frac{4x^3 8x^7}{1 - x^4 + x^8}$$

para $x < 1$ y si $x = 0.25$. Escriba y ejecute un algoritmo que determine el número de términos necesarios en el lado izquierdo de la ecuación de tal forma que el lado izquierdo difiera del lado derecho en menos de 10^{-6}

```
In [23]: def lado_derecho(x):
    return (1 + 2 * x) / (1 + x + x**2)

def lado_izquierdo(x, tolerancia):
```



```

suma = 0.0
n = 0
error = 1.0
valor_real = lado_derecho(x)

A = 1
y = x**2

while error > tolerancia:
    numerador = (A * (1 - 2 * x))
    denominador = (1 - x + x**2)
    termino = numerador / denominador

    suma += termino
    n += 1

    error = abs(suma - valor_real)

    A *= 2 * x
    x *= x

return n, suma, valor_real

x = 0.25
tolerancia = 1e-6

n, suma, derecho = lado_izquierdo(x, tolerancia)
print(f"Número de términos necesarios: {n}")
print(f"Lado izquierdo (suma parcial): {suma}")
print(f"Lado derecho (valor real): {derecho}")

```

Número de términos necesarios: 4
 Lado izquierdo (suma parcial): 1.1428571279559818
 Lado derecho (valor real): 1.1428571428571428